MATH 114 HW 6

Dani Nguyen

Winter 2026

Question 1
We have

$$\begin{aligned}
dX &= -\frac{D}{4X^3}dt + \frac{1}{2X}\sqrt{2D}dB, \quad X(0) = x_0 \\
d(f(X)) &= \left(uf' + \frac{1}{2}\sigma^2 f''\right)dt + f'\sigma dB \\
d(X^2) &= \left(-\frac{D}{4X^3}\cdot 2X + \frac{1}{2}\cdot\frac{1}{4X^2}2D\cdot 2\right)dt + 2X\frac{1}{2X}\sqrt{2D}dB \\
&= \left(-\frac{D}{2X^2} + \frac{D}{2X^2}\right)dt + \sqrt{2D}dB \\
&= \sqrt{2D}dB
\end{aligned}$$

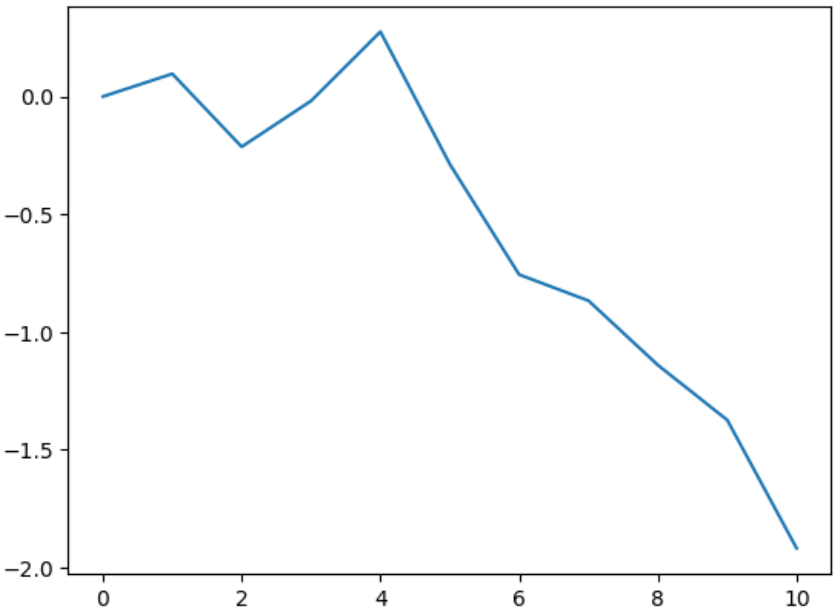
Let $y(t) = X^2$

$$\begin{aligned}
dy &= \sqrt{2D}dB \\
y &= \sqrt{2D}B(t) \\
X^2 &\sim N(0, 2Dt)
\end{aligned}$$

Question 2

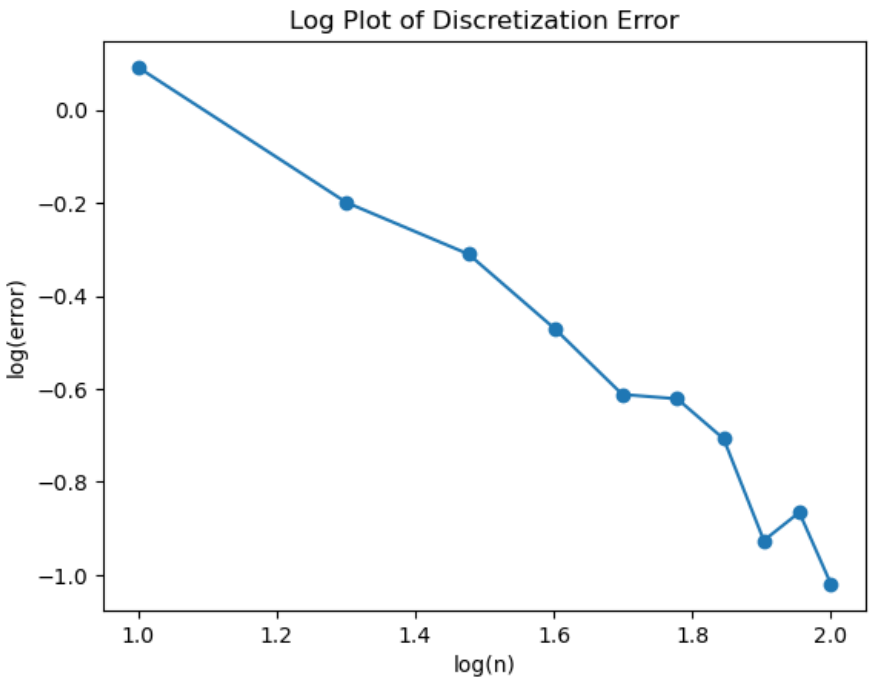
a.

X_0	0
X_1	0.09636
X_2	-0.21323
X_3	-0.01857
X_4	0.27514
X_5	-0.28679
X_6	-0.75594
X_7	-0.86670
X_8	-1.14005
X_9	-1.37337
X_{10}	-1.91780



b. Repeating part a. 10,000 times, we get 6.15255 as the approximate value of $\mathbb{E}(X(1))$

c. Repeating part a. and b. for $n = 10, \ldots, 100$, we get



We see $\log f(n)$ decrease logarithmically as we increase the log of the number of discretization steps.

Below is the code for my simulations.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

def euler(T, n, output_index=-1):
    X = [1]
    delta_t = T / n
    for i in np.arange(1, n+1):
        X.append(X[i-1] + 2 * X[i-1] * delta_t + np.sqrt(delta_t)
        * rng.normal())
    return X[output_index]

def euler_sample_mean(T, n, num_samples):
    samples = []
```

```
    for i in range(num_samples):
        samples.append(euler(T, n))
    return np.mean(samples)

def discretization_error(T, num_samples):
    fn = []
    for n in np.arange(10,101,10):
        fn.append(np.abs(euler_sample_mean(T, n, num_samples)
        - np.exp(2)))
    return fn

discretization = discretization_error(1, 10000)
X = np.log10(np.arange(10,101,10))
Y = np.log10(discretization)

plt.plot(X, Y, marker='o')
plt.xlabel('log(n)')
plt.ylabel('log(error)')
plt.title('Log Plot of Discretization Error')
```